

Assignment 1: Write Python scripts to implement basic operations (addition, subtraction, multiplication, matrix multiplication)

```
import tensorflow as tf

tensor_1 = tf.constant([[1, 2], [3, 4]])
tensor_2 = tf.constant([[5, 6], [7, 8]])

print("Tensor 1:\n", tensor_1)
print("Tensor 2:\n", tensor_2)

add = tf.add(tensor_1, tensor_2)
sub = tf.subtract(tensor_1, tensor_2)
mul = tf.multiply(tensor_1, tensor_2)
matmul = tf.matmul(tensor_1, tensor_2)

print("\nAddition:\n", add)
print("\nSubtraction:\n", sub)
print("\nElement-wise Multiplication:\n", mul)
print("\nMatrix Multiplication:\n", matmul)
```

Assignment 2: Write Python scripts to perform reshaping and slicing operations on tensors.

```
import tensorflow as tf

tensor_3 = tf.constant([1, 2, 3, 4, 5, 6])

print("Original Tensor:\n", tensor_3)

reshaped = tf.reshape(tensor_3, [2, 3])
```

```
print("\nReshaped Tensor (2x3):\n", reshaped)
```

```
sliced = reshaped[:, 1:]
```

```
print("\nSliced Tensor (columns 2 and 3):\n", sliced)
```

Assignment 3: Write Python scripts to do type conversion, using tensor flow and print tensor properties (shape, dtype, etc.)

```
import tensorflow as tf
```

```
tensor_1 = tf.constant([[1, 2], [3, 4]])
```

```
print("Original Tensor:\n", tensor_1)
```

```
float_tensor = tf.cast(tensor_1, dtype=tf.float32)
```

```
print("\nFloat Tensor:\n", float_tensor)
```

```
#Working with tf.Variable (mutable tensor)
```

```
var = tf.Variable([[1, 2], [3, 4]])
```

```
print("\nOriginal Variable Tensor:\n", var)
```

```
# Updating variable values
```

```
var.assign_add([[10, 10], [10, 10]])
```

```
print("\nUpdated Variable Tensor:\n", var)
```

```
print("\nShape of tensor_1:", tensor_1.shape)
```

```
print("Data type of tensor_1:", tensor_1.dtype)
```

Assignment 4: Write Python scripts to handle missing values (dropping and filling) using python

libraries such as Pandas and NumPy

```
import pandas as pd

import numpy as np

data = {
    'Name': ['Alice', 'Bob', 'Charlie', 'David', np.nan],
    'Age': [25, 30, np.nan, 40, 35],
    'Salary': [50000, 60000, 55000, np.nan, 70000],
    'Department': ['HR', 'IT', 'HR', 'Finance', 'IT']
}

df = pd.DataFrame(data)

print("Original Dataset:\n", df)


df_dropna = df.dropna()

print("\nDataset after dropping missing values:\n", df_dropna)


df_filled = df.copy()

df_filled['Age'] = df_filled['Age'].fillna(df_filled['Age'].mean())

df_filled['Salary'] = df_filled['Salary'].fillna(df_filled['Salary'].mean())

df_filled['Name'] = df_filled['Name'].fillna(df_filled['Name'].mode()[0])

print("\nDataset after filling missing values:\n", df_filled)
```

Assignment 5: Write Python scripts to apply normalization (Min-Max scaling) and standardization

(Z-score) on numeric features using python libraries.

```
import pandas as pd
```

```
from sklearn.preprocessing import MinMaxScaler, StandardScaler
```

```
# Create data
```

```
data = {  
    "Name": ["Alice", "Bob", "Charlie", "David", "Eve"],  
    "Age": [25, 30, 28, 40, 35],  
    "Salary": [50000, 60000, 55000, 65000, 70000]  
}
```

```
df = pd.DataFrame(data)
```

```
print("Original Data:")
```

```
print(df)
```

```
# Min-Max Normalization
```

```
minmax = MinMaxScaler()
```

```
df[["Age", "Salary"]] = minmax.fit_transform(df[["Age", "Salary"]])
```

```
print("\nMin-Max Normalized Data:")
```

```
print(df)
```

```
# Z-Score Standardization
```

```
standard = StandardScaler()
```

```
df[["Age", "Salary"]] = standard.fit_transform(df[["Age", "Salary"]])

print("\nZ-Score Standardized Data:")

print(df)
```

Assignment 6: Write Python scripts to encode categorical variables using Label Encoding and One-Hot Encoding using python libraries

```
import pandas as pd

from sklearn.preprocessing import LabelEncoder
```

```
data = {
    'Name': ['Alice', 'Bob', 'Charlie', 'David', 'Eve'],
    'Department': ['HR', 'IT', 'Finance', 'HR', 'IT'],
    'Gender': ['F', 'M', 'M', 'M', 'F']
}

df = pd.DataFrame(data)

print("Original Dataset:\n", df)
```

```
#Apply Label Encoding

le = LabelEncoder()

df_label = df.copy()

df_label['Department_Label'] = le.fit_transform(df_label['Department'])

df_label['Gender_Label'] = le.fit_transform(df_label['Gender'])

print("\nLabel Encoded Data:\n", df_label)
```

```
#Apply One-Hot Encoding

df_onehot = pd.get_dummies(df, columns=['Department', 'Gender'])
```

```
print("\nOne-Hot Encoded Data:\n", df_onehot)
```

Assignment 7: Write Python scripts to plot a histogram for numerical data distribution using Matplotlib or Seaborn libraries.

```
import pandas as pd
```

```
import seaborn as sns
```

```
import matplotlib.pyplot as plt
```

```
data = {  
'Student': ['A', 'B', 'C', 'D', 'E'],  
'Math': [85, 78, 92, 70, 88],  
'Science': [80, 75, 89, 65, 90],  
'English': [78, 82, 88, 72, 85]  
}
```

```
df = pd.DataFrame(data)
```

```
print("Dataset:\n", df)
```

```
#Plot a histogram for Math scores
```

```
sns.histplot(df['Math'], kde=True, color='orange')
```

```
plt.title("Histogram of Math Scores")
```

```
plt.xlabel("Math Score")
```

```
plt.ylabel("Number of Students")
```

```
plt.show()
```

Assignment 8: Write Python scripts to plot a scatter plot for showing relationship between two variables using Matplotlib or Seaborn libraries.

```

import pandas as pd

import seaborn as sns

import matplotlib.pyplot as plt

data = {
    'Student': ['A', 'B', 'C', 'D', 'E'],
    'Math': [85, 78, 92, 70, 88],
    'Science': [80, 75, 89, 65, 90],
    'English': [78, 82, 88, 72, 85]
}

df = pd.DataFrame(data)

print("Dataset:\n", df)

#Create a scatter plot (Math vs Science)

sns.scatterplot(x='Math', y='Science', data=df, s=80, color='green', marker='o')

plt.title("Scatter Plot - Math vs Science Scores")

plt.xlabel("Math Score")

plt.ylabel("Science Score")

plt.show()

```

Assignment 9: Write Python scripts to create a heatmap of correlation values between multiple variables using Matplotlib or Seaborn libraries.

```

import pandas as pd

import seaborn as sns

import matplotlib.pyplot as plt

```

```

data = {
    'Student': ['A', 'B', 'C', 'D', 'E'],
    'Math': [85, 78, 92, 70, 88],
    'Science': [80, 75, 89, 65, 90],
    'English': [78, 82, 88, 72, 85]
}

df = pd.DataFrame(data)

print("Dataset:\n", df)


#Calculate correlation (only for numeric columns)

corr_matrix = df.drop('Student', axis=1).corr()

print("\nCorrelation Matrix:\n", corr_matrix)


#Create a heatmap to visualize correlations

sns.heatmap(corr_matrix, annot=True, cmap='YlGnBu', linewidths=0.5)

plt.title("Correlation Heatmap of Subject Scores")

plt.show()

```

Assignment 10: Develop applications for text generation tasks such as story generation using trained Generative AI models.

```

from transformers import GPT2LMHeadModel, GPT2Tokenizer

import torch


# Step 1: Load GPT-2 Model and Tokenizer

model_name = "gpt2"

```

```
tokenizer = GPT2Tokenizer.from_pretrained(model_name)
model = GPT2LMHeadModel.from_pretrained(model_name)
```

Step 2: Configure Padding

```
tokenizer.pad_token = tokenizer.eos_token
```

Step 3: Prepare Input Prompt

```
prompt = "Once upon a time in a distant galaxy,"
```

```
inputs = tokenizer(
    prompt,
    return_tensors="pt",
    padding=True
)
```

Step 4: Generate Text

```
outputs = model.generate(
    inputs["input_ids"],
    attention_mask=inputs["attention_mask"],
    max_length=100,
    temperature=0.8,
    top_k=50,
    top_p=0.95,
    repetition_penalty=1.2,
    pad_token_id=tokenizer.eos_token_id,
    num_return_sequences=1,
```

```
        do_sample=True
    )

# Step 5: Decode Output
generated_text = tokenizer.decode(
    outputs[0],
    skip_special_tokens=True
)
```

```
# Step 6: Display Result
print("Generated Story:\n")
print(generated_text)
```

OR

```
from transformers import GPT2LMHeadModel, GPT2Tokenizer
```

```
# Load GPT-2
```

```
tokenizer = GPT2Tokenizer.from_pretrained("gpt2")
```

```
model = GPT2LMHeadModel.from_pretrained("gpt2")
```

```
# Input text
```

```
text = "Once upon a time in a distant galaxy,"
```

```
# Convert text into tokens
```

```
inputs = tokenizer(text, return_tensors="pt")
```

```
# Generate text

output = model.generate(

    inputs["input_ids"],

    max_length=100,

    do_sample=True

)


# Convert tokens back to text

result = tokenizer.decode(output[0], skip_special_tokens=True)


print("Generated Text:")

print(result)
```

Assignment 11: Develop applications for text generation tasks such as dialogue generation using trained Generative AI models.

Step 1: Import Required Libraries

```
from transformers import AutoModelForCausalLM, AutoTokenizer

import torch
```

AutoTokenizer → Converts text into tokens.

AutoModelForCausalLM → Loads a causal language model for text generation.

Step 2: Load Pre-trained Conversational Model (DialoGPT)

```
model_name = "microsoft/DialoGPT-medium"
```

```
tokenizer = AutoTokenizer.from_pretrained(model_name)
```

```
model = AutoModelForCausalLM.from_pretrained(model_name)
```

```
# Step 3: Initialize Chat History
```

```
chat_history_ids = None
```

```
# Step 4: Start Chatbot Interaction
```

```
print("English Chatbot is ready! Type 'exit' to quit.\n")
```

```
# Step 5: Chat Loop (5 dialogue exchanges)
```

```
for step in range(5):
```

```
    user_input = input("You: ")
```

```
    if user_input.lower() == "exit":
```

```
        print("Chat ended.")
```

```
        break
```

```
# Encode user's input and add end-of-sequence token
```

```
new_input_ids = tokenizer.encode(
```

```
    user_input + tokenizer.eos_token,
```

```
    return_tensors="pt"
```

```
)
```

```
# Concatenate with conversation history
```

```
if chat_history_ids is not None:
```

```
    bot_input_ids = torch.cat(
```

```

        [chat_history_ids, new_input_ids],
        dim=-1
    )
else:
    bot_input_ids = new_input_ids

# Step 6: Generate Bot Response
chat_history_ids = model.generate(
    bot_input_ids,
    max_length=1000,
    pad_token_id=tokenizer.eos_token_id,
    do_sample=True,
    top_k=50,
    top_p=0.92,
    temperature=0.7,
    repetition_penalty=1.2
)

```

```

# Step 7: Decode Bot's Reply
response = tokenizer.decode(
    chat_history_ids[:, bot_input_ids.shape[-1]:][0],
    skip_special_tokens=True
)

```

```

print("Bot:", response)

```

OR

```
from transformers import AutoModelForCausalLM, AutoTokenizer  
import torch
```

```
# Load model
```

```
model_name = "microsoft/DialoGPT-medium"
```

```
tokenizer = AutoTokenizer.from_pretrained(model_name)
```

```
model = AutoModelForCausalLM.from_pretrained(model_name)
```

```
chat_history_ids = None
```

```
print("English Chatbot is ready! Type 'exit' to quit.\n")
```

```
for step in range(5):
```

```
    user_input = input("You: ")
```

```
    if user_input.lower() == "exit":
```

```
        print("Chat ended.")
```

```
        break
```

```
    new_input = tokenizer.encode(
```

```
        user_input + tokenizer.eos_token,
```

```
        return_tensors="pt"
```

```
    )
```

```
    bot_input = torch.cat(
```

```
        [chat_history_ids, new_input], dim=-1
```

```
) if chat_history_ids is not None else new_input
```

```
chat_history_ids = model.generate(  
    bot_input,  
    max_length=1000,  
    pad_token_id=tokenizer.eos_token_id,  
    do_sample=True,  
    top_k=50,  
    top_p=0.92,  
    temperature=0.7  
)
```

```
response = tokenizer.decode(  
    chat_history_ids[:, bot_input.shape[-1]:][0],  
    skip_special_tokens=True  
)
```

```
print("Bot:", response)
```

Assignment 12: Text Generation: Implement a Long Short-Term Memory (LSTM) network using TensorFlow 2 for text generation tasks. Train the LSTM model on a dataset of text sequences and generate new text samples

```
import numpy as np
```

```
import tensorflow as tf
```

```
# =====
```

```
# 1. Download Shakespeare Dataset
```

```
# =====
```

```
path = tf.keras.utils.get_file(  
    "shakespeare.txt",  
    "https://storage.googleapis.com/download.tensorflow.org/data/shakespeare.txt"  
)
```

```
# Read the dataset
```

```
text = open(path, "r", encoding="utf-8").read()[:100000]
```

```
print("Dataset loaded successfully!")
```

```
print("Number of characters:", len(text))
```

```
# =====
```

```
# 2. Tokenization
```

```
# =====
```

```
tok = tf.keras.preprocessing.text.Tokenizer()
```

```
# Create word-to-index mapping
```

```
tok.fit_on_texts([text])
```

```
# Convert text into word indices
```

```
seq = tok.texts_to_sequences([text])[0]
```

```
# Vocabulary size
```

```
vocab = len(tok.word_index) + 1
```

```
print("Vocabulary size:", vocab)
```

```
# =====
```

```
# 3. Create Input and Output Sequences
```

```
# =====
```

```
seq_len = 10
```

```
X = []
```

```
y = []
```

```
for i in range(len(seq) - seq_len):
```

```
    # 10 words as input
```

```
    X.append(seq[i:i + seq_len])
```

```

# 11th word as target
y.append(seq[i + seq_len])

# Use only first 10,000 samples
X = np.array(X[:10000])
y = np.array(y[:10000])

print("X shape:", X.shape)
print("y shape:", y.shape)

# =====

# 4. One-Hot Encoding
# =====

y = tf.keras.utils.to_categorical(
    y,
    num_classes=vocab
)

# =====

# 5. Build LSTM Model
# =====

model = tf.keras.Sequential([

```

```

# Converts word IDs into 32-dimensional vectors

tf.keras.layers.Embedding(

    input_dim=vocab,

    output_dim=32

),

# LSTM layer

tf.keras.layers.LSTM(64),

# Output layer

tf.keras.layers.Dense(

    vocab,

    activation="softmax"

)

])

```

```

# =====

```

```

# 6. Compile Model

```

```

# =====

```

```

model.compile(

    loss="categorical_crossentropy",

    optimizer="adam",

    metrics=["accuracy"]
)

```

```
)
```

```
# Display model architecture
```

```
model.summary()
```

```
# =====
```

```
# 7. Train Model
```

```
# =====
```

```
model.fit(
```

```
    X,
```

```
    y,
```

```
    epochs=5,
```

```
    batch_size=128
```

```
)
```

```
# =====
```

```
# 8. Text Generation Function
```

```
# =====
```

```
def generate(start="BRUTUS", length=50, temp=0.7):
```

```
    # Convert starting text into word IDs
```

```
inp = tok.texts_to_sequences([start])[0]

# Keep only last seq_len words
inp = inp[-seq_len:]

# Padding if input has fewer than seq_len words
inp = [0] * (seq_len - len(inp)) + inp

# Add batch dimension
inp = tf.expand_dims(inp, 0)

# Store generated words
out = []

# Generate words
for _ in range(length):

    # Predict next word
    p = model.predict(inp, verbose=0)[0]

    # Apply temperature
    p = np.asarray(p).astype("float64")

    p = np.log(p + 1e-8) / temp
    p = np.exp(p)
```

```
# Normalize probabilities
```

```
p = p / np.sum(p)
```

```
# Select a word according to probability
```

```
w_id = np.random.choice(
```

```
    vocab,
```

```
    p=p
```

```
)
```

```
# Convert ID back to word
```

```
word = tok.index_word.get(w_id, "")
```

```
# Add word to output
```

```
if word:
```

```
    out.append(word)
```

```
# Update input sequence
```

```
new_input = list(inp[0].numpy()[1:])
```

```
new_input.append(w_id)
```

```
inp = tf.expand_dims(
```

```
    new_input,
```

```
    0
```

```
)
```

```
# Return generated sentence
```

```

return start + " " + " ".join(out)

# =====

# 9. Generate Text

# =====

print("\nGenerated Text:")
print("-----")

result = generate(
    start="BRUTUS",
    length=50,
    temp=0.7
)

print(result)

```

OR

```

import numpy as np
import tensorflow as tf

# Download dataset
path = tf.keras.utils.get_file(
    "shakespeare.txt",
    "https://storage.googleapis.com/download.tensorflow.org/data/shakespeare.txt"
)

```

```
)
```

```
# Read text
```

```
text = open(path, "r", encoding="utf-8").read()[:50000]
```

```
# Tokenization
```

```
tokenizer = tf.keras.preprocessing.text.Tokenizer()
```

```
tokenizer.fit_on_texts([text])
```

```
sequence = tokenizer.texts_to_sequences([text])[0]
```

```
vocab_size = len(tokenizer.word_index) + 1
```

```
# Create input and output
```

```
seq_len = 5
```

```
X = []
```

```
y = []
```

```
for i in range(len(sequence) - seq_len):
```

```
    X.append(sequence[i:i + seq_len])
```

```
    y.append(sequence[i + seq_len])
```

```
# Use small dataset
```

```
X = np.array(X[:5000])
```

```
y = np.array(y[:5000])
```

```
# Convert output to one-hot encoding
```

```
y = tf.keras.utils.to_categorical(y, num_classes=vocab_size)
```

```
# Create LSTM model
```

```
model = tf.keras.Sequential([  
    tf.keras.layers.Embedding(vocab_size, 32),  
    tf.keras.layers.LSTM(64),  
    tf.keras.layers.Dense(vocab_size, activation="softmax")  
])
```

```
# Compile model
```

```
model.compile(  
    optimizer="adam",  
    loss="categorical_crossentropy",  
    metrics=["accuracy"]  
)
```

```
# Train model
```

```
model.fit(X, y, epochs=3, batch_size=64)
```

```
# Generate text
```

```
def generate_text(start_word, length=20):
```

```
    words = tokenizer.texts_to_sequences([start_word])[0]
```

```
    words = [0] * (seq_len - len(words)) + words
```

```
    words = words[-seq_len:]
```

```
result = start_word
```

```
for i in range(length):
```

```
    prediction = model.predict(  
        np.array([words]),  
        verbose=0  
    )[0]
```

```
    next_word = np.argmax(prediction)
```

```
    word = tokenizer.index_word.get(next_word, "")
```

```
    result += " " + word
```

```
    words = words[1:] + [next_word]
```

```
return result
```

```
# Generate and print text
```

```
print("\nGenerated Text:")
```

```
print(generate_text("brutus", 20))
```

Assignment 13: Implement Variational Auto encoders (VAE) using Tensorflow for image generation

```
# Import libraries

import numpy as np

import tensorflow as tf

import matplotlib.pyplot as plt

from tensorflow.keras import layers

from tensorflow.keras.datasets import mnist


# Load MNIST dataset

(x_train, _), (x_test, _) = mnist.load_data()


# Normalize images

x_train = x_train.astype("float32") / 255.0

x_test = x_test.astype("float32") / 255.0


# Convert 28x28 images into 784 values

x_train = x_train.reshape(-1, 784)

x_test = x_test.reshape(-1, 784)


# Latent space size

latent_dim = 2
```

```

# -----

# Encoder

# -----

encoder_input = tf.keras.Input(shape=(784,))

x = layers.Dense(256, activation="relu")(encoder_input)

z_mean = layers.Dense(latent_dim)(x)
z_log_var = layers.Dense(latent_dim)(x)


# Sampling function
def sampling(args):
    z_mean, z_log_var = args

    epsilon = tf.random.normal(
        shape=tf.shape(z_mean)
    )

    return z_mean + tf.exp(0.5 * z_log_var) * epsilon


z = layers.Lambda(sampling)(
    [z_mean, z_log_var]
)

```

```
encoder = tf.keras.Model(  
    encoder_input,  
    [z_mean, z_log_var, z],  
    name="encoder"  
)
```

```
# -----
```

```
# Decoder
```

```
# -----
```

```
decoder_input = tf.keras.Input(  
    shape=(latent_dim,) )
```

```
x = layers.Dense(  
    256,  
    activation="relu"  
) (decoder_input)
```

```
decoder_output = layers.Dense(  
    784,  
    activation="sigmoid"  
) (x)
```

```
decoder = tf.keras.Model(  
    decoder_input,  
    decoder_output,  
    name="decoder"  
)
```

```
# -----
```

```
# VAE Model
```

```
# -----
```

```
class VAE(tf.keras.Model):
```

```
    def __init__(self, encoder, decoder):
```

```
        super().__init__()
```

```
        self.encoder = encoder
```

```
        self.decoder = decoder
```

```
    def train_step(self, data):
```

```
        if isinstance(data, tuple):
```

```
            data = data[0]
```

```
        with tf.GradientTape() as tape:
```

```
z_mean, z_log_var, z = self.encoder(data)
```

```
reconstruction = self.decoder(z)
```

```
# Reconstruction loss
```

```
recon_loss = tf.reduce_mean(  
    tf.reduce_sum(  
        tf.keras.losses.binary_crossentropy(  
            data,  
            reconstruction  
        ),  
        axis=-1  
    )  
)
```

```
# KL divergence loss
```

```
kl_loss = -0.5 * tf.reduce_mean(  
    tf.reduce_sum(  
        1 + z_log_var  
        - tf.square(z_mean)  
        - tf.exp(z_log_var),  
        axis=1  
    )  
)
```

```
# Total loss
```

```
total_loss = recon_loss + kl_loss
```

```
# Calculate gradients
```

```
gradients = tape.gradient(
```

```
    total_loss,
```

```
    self.trainable_weights
```

```
)
```

```
# Update weights
```

```
self.optimizer.apply_gradients(
```

```
    zip(gradients, self.trainable_weights)
```

```
)
```

```
return {
```

```
    "loss": total_loss
```

```
}
```

```
# -----
```

```
# Train VAE
```

```
# -----
```

```
vae = VAE(encoder, decoder)
```

```
vae.compile(
```

```
    optimizer="adam"
```

```
)
```

```
vae.fit(  
    x_train,  
    x_train,  
    epochs=10,  
    batch_size=128  
)
```

```
# -----  
# Generate New Digits  
# -----
```

```
def generate_digits(decoder):  
  
    n = 10  
    digit_size = 28  
  
    grid = np.zeros(  
        (digit_size * n, digit_size * n)  
    )  
  
    for i, y in enumerate(  
        np.linspace(-2, 2, n)  
    ):
```

```
for j, x in enumerate(  
    np.linspace(-2, 2, n)  
):
```

```
    z_sample = np.array([[x, y]])
```

```
    generated = decoder.predict(  
        z_sample,  
        verbose=0  
    )
```

```
    digit = generated[0].reshape(  
        digit_size,  
        digit_size  
    )
```

```
    grid[  
        i * digit_size:(i + 1) * digit_size,  
        j * digit_size:(j + 1) * digit_size  
    ] = digit
```

```
# Display generated digits
```

```
plt.figure(figsize=(8, 8))
```

```
plt.imshow(  
    grid,  
    cmap=cm.gray,  
    interpolation='nearest',  
    vmin=0, vmax=1,  
    extent=[0, 10, 0, 10])  
plt.show()
```

```
    grid,  
    cmap="gray"  
)
```

```
plt.axis("off")
```

```
plt.show()
```

```
# Generate digits
```

```
generate_digits(decoder)
```

OR

```
import numpy as np
```

```
import tensorflow as tf
```

```
import matplotlib.pyplot as plt
```

```
from tensorflow.keras import layers
```

```
from tensorflow.keras.datasets import mnist
```

```
# Load MNIST
```

```
(x_train, _), (_, _) = mnist.load_data()
```

```
# Normalize data
```

```
x_train = x_train.astype("float32") / 255.0
```

```
# Convert 28x28 to 784
```

```
x_train = x_train.reshape(-1, 784)
```

```
# Latent dimension
```

```
latent_dim = 2
```

```
# Encoder
```

```
encoder_input = layers.Input(shape=(784,))
```

```
x = layers.Dense(128, activation="relu")(encoder_input)
```

```
z_mean = layers.Dense(latent_dim)(x)
```

```
z_log_var = layers.Dense(latent_dim)(x)
```

```
encoder = tf.keras.Model(
```

```
    encoder_input,
```

```
    [z_mean, z_log_var]
```

```
)
```

```
# Decoder
```

```
decoder_input = layers.Input(shape=(latent_dim,))
```

```
x = layers.Dense(128, activation="relu")(decoder_input)
```

```
decoder_output = layers.Dense(
```

```
    784,
```

```
    activation="sigmoid"
```

```
)(x)
```

```
decoder = tf.keras.Model(  
    decoder_input,  
    decoder_output  
)
```

Simple VAE

```
class VAE(tf.keras.Model):
```

```
    def __init__(self, encoder, decoder):
```

```
        super().__init__()
```

```
        self.encoder = encoder
```

```
        self.decoder = decoder
```

```
    def train_step(self, data):
```

```
        with tf.GradientTape() as tape:
```

```
            mean, log_var = self.encoder(data)
```

```
            # Random sampling
```

```
            epsilon = tf.random.normal(  
                shape=tf.shape(mean)
```

```
            )
```

```
            z = mean + tf.exp(
```

```

        0.5 * log_var
    ) * epsilon

output = self.decoder(z)

# Reconstruction loss
loss1 = tf.reduce_mean(
    tf.keras.losses.binary_crossentropy(
        data, output
    )
)

# KL loss
loss2 = -0.5 * tf.reduce_mean(
    1 + log_var
    - tf.square(mean)
    - tf.exp(log_var)
)

loss = loss1 + loss2

gradients = tape.gradient(
    loss,
    self.trainable_weights
)

```

```
self.optimizer.apply_gradients(  
    zip(gradients, self.trainable_weights)  
)
```

```
return {"loss": loss}
```

```
# Create and train VAE
```

```
vae = VAE(encoder, decoder)
```

```
vae.compile(optimizer="adam")
```

```
vae.fit(  
    x_train,  
    epochs=5,  
    batch_size=128  
)
```

```
# Generate a new digit
```

```
z = np.random.normal(  
    size=(1, latent_dim)  
)
```

```
generated = decoder.predict(  
    z,
```

```
        verbose=0
    )

    digit = generated[0].reshape(28, 28)

    # Display digit
    plt.imshow(
        digit,
        cmap="gray"
    )

    plt.axis("off")
    plt.show()
```

Assignment 14: Text generation: Implement a Transformer-based language model (e.g., GPT) using TensorFlow 2 for text generation. Fine-tune the model on a text corpus and generate coherent and contextually relevant text.

```
# Import libraries

import numpy as np

import tensorflow as tf

from tensorflow.keras.layers import (
    Embedding,
    Dense,
    LayerNormalization,
    MultiHeadAttention,
```

Dropout

)

```
from tensorflow.keras.models import Model
```

```
# =====
```

```
# 1. Load Shakespeare Dataset
```

```
# =====
```

```
path = tf.keras.utils.get_file(
    "shakespeare.txt",
    "https://storage.googleapis.com/download.tensorflow.org/data/shakespeare.txt"
)
```

```
text = open(
    path,
    "r",
    encoding="utf-8"
).read()[:20000]
```

```
print("Dataset loaded successfully!")
```

```
# =====
```

```
# 2. Tokenization
```

```
# =====
```

```
tokenizer = tf.keras.preprocessing.text.Tokenizer(  
    oov_token="<OOV>"  
)
```

```
tokenizer.fit_on_texts([text])
```

```
sequence = tokenizer.texts_to_sequences([text])[0]
```

```
vocab = len(tokenizer.word_index) + 1
```

```
print("Vocabulary size:", vocab)
```

```
# =====
```

```
# 3. Create Training Data
```

```
# =====
```

```
seq_len = 10
```

```
X = []
```

```
y = []
```

```
for i in range(len(sequence) - seq_len):
```

```
X.append(  
    sequence[i:i + seq_len]  
)
```

```
y.append(  
    sequence[i + seq_len]  
)
```

```
# Use only 4000 samples
```

```
X = np.array(X[:4000])
```

```
y = np.array(y[:4000])
```

```
print("X shape:", X.shape)
```

```
print("y shape:", y.shape)
```

```
# Create dataset
```

```
dataset = tf.data.Dataset.from_tensor_slices(  
    (X, y)  
)
```

```
dataset = dataset.shuffle(4000).batch(32)
```

```
# =====
```

4. Positional Encoding

```
# =====
```

```
def positional_encoding(length, depth):
```

```
    position = np.arange(length)[:, np.newaxis]
```

```
    dimension = np.arange(depth)[np.newaxis, :]
```

```
    angle = position / np.power(
        10000,
        (2 * (dimension // 2)) / depth
    )
```

```
    encoding = np.zeros(
        (length, depth)
    )
```

```
    encoding[:, 0::2] = np.sin(
        angle[:, 0::2]
    )
```

```
    encoding[:, 1::2] = np.cos(
        angle[:, 1::2]
    )
```

```
return tf.cast(
    encoding,
    tf.float32
)
```

```
# =====
```

```
# 5. Transformer Block
```

```
# =====
```

```
class TransformerBlock(
    tf.keras.layers.Layer
):
```

```
def __init__(
    self,
    dim,
    heads,
    ff_dim
):
```

```
    super().__init__()
```

```
    self.attention = MultiHeadAttention(
        num_heads=heads,
        key_dim=dim
```

```
)
```

```
self.dense1 = Dense(  
    ff_dim,  
    activation="relu"  
)
```

```
self.dense2 = Dense(dim)
```

```
self.norm1 = LayerNormalization()
```

```
self.norm2 = LayerNormalization()
```

```
self.dropout = Dropout(0.1)
```

```
def call(  
    self,  
    x,  
    training=False  
):
```

```
    # Self attention
```

```
    attention_output = self.attention(  
        x,  
        x
```

```
)
```

```
attention_output = self.dropout(  
    attention_output,  
    training=training  
)
```

```
# Residual connection  
x = self.norm1(  
    x + attention_output  
)
```

```
# Feed forward network  
ff_output = self.dense1(x)
```

```
ff_output = self.dense2(  
    ff_output  
)
```

```
ff_output = self.dropout(  
    ff_output,  
    training=training  
)
```

```
# Residual connection  
x = self.norm2(  

```

```
        x + ff_output
    )
```

```
    return x
```

```
# =====
```

```
# 6. Mini GPT Model
```

```
# =====
```

```
class MiniGPT(Model):
```

```
    def __init__(
```

```
        self,
```

```
        vocab,
```

```
        maxlen
```

```
    ):
```

```
        super().__init__()
```

```
        # Word embedding
```

```
        self.embedding = Embedding(
```

```
            vocab,
```

```
            64
```

```
        )
```

```
# Positional encoding
```

```
self.position = positional_encoding(  
    maxlen,  
    64  
)
```

```
# Transformer block
```

```
self.transformer = TransformerBlock(  
    64,  
    2,  
    128  
)
```

```
# IMPORTANT:
```

```
# Do NOT use self.output
```

```
# because Keras already has an output property.
```

```
self.output_layer = Dense(  
    vocab  
)
```

```
def call(  
    self,  
    x,  
    training=False
```

);

Word embedding

x = self.embedding(x)

Add positional information

x = x + self.position[

tf.shape(x)[1],

:

]

Transformer

x = self.transformer(

x,

training=training

)

Use only last word

x = x[:, -1, :]

Predict next word

return self.output_layer(x)

=====

7. Create Model

```
# =====
```

```
model = MiniGPT(  
    vocab,  
    seq_len  
)
```

```
# =====
```

```
# 8. Compile Model
```

```
# =====
```

```
model.compile(  
    optimizer="adam",  
    loss=tf.keras.losses.SparseCategoricalCrossentropy(  
        from_logits=True  
    ),  
    metrics=["accuracy"]  
)
```

```
# =====
```

```
# 9. Train Model
```

```
# =====
```

```
model.fit(  

```

```

dataset,

epochs=5
)

# =====

# 10. Generate Text

# =====

def generate_text(
    seed,
    steps=20,
    temperature=1.0
):

    result = seed

    for i in range(steps):

        # Convert text to numbers

        tokens = tokenizer.texts_to_sequences(
            [result]
        )[0]

        # Take last 10 words

        tokens = tokens[-seq_len:]

```

```
# Padding
```

```
tokens = tf.keras.preprocessing.sequence.pad_sequences(  
    [tokens],  
    maxlen=seq_len  
)
```

```
# Prediction
```

```
logits = model(  
    tokens,  
    training=False  
)[0]
```

```
# Temperature
```

```
logits = logits / temperature
```

```
# Convert to probabilities
```

```
probabilities = tf.nn.softmax(  
    logits  
).numpy()
```

```
# Select next word
```

```
next_id = np.random.choice(  
    len(probabilities),  
    p=probabilities  
)
```

```

# Convert ID to word

word = tokenizer.index_word.get(
    next_id,
    ""
)

# Add word

if word:
    result += " " + word

return result


# =====

# 11. Generate Example

# =====

print("\nGenerated Text:")
print("-----")

print(
    generate_text(
        "To be or not",
        steps=20,
        temperature=1.0

```

```
)  
)
```

Assignment 15: Implement a Generative Adversarial Network (GAN) architecture using TensorFlow 2. Train the GAN model on a dataset such as MNIST or CIFAR-10 for image generation tasks.

```
# Import libraries
```

```
import tensorflow as tf
```

```
from tensorflow.keras import layers
```

```
import matplotlib.pyplot as plt
```

```
import numpy as np
```

```
# =====
```

```
# 1. Load MNIST Dataset
```

```
# =====
```

```
(train_images, _), (_, _) =  
tf.keras.datasets.mnist.load_data()
```

```
# Reshape images to 28x28x1
```

```
train_images = train_images.reshape(  
    train_images.shape[0], 28, 28, 1  
) .astype("float32")
```

```
# Normalize images to -1 to 1
```

```
train_images = (train_images - 127.5) / 127.5
```

```
# Create dataset
```

```
BUFFER_SIZE = 60000
```

```
BATCH_SIZE = 256
```

```
dataset = tf.data.Dataset.from_tensor_slices(  
    train_images  
)
```

```
dataset = dataset.shuffle(  
    BUFFER_SIZE  
) . batch(BATCH_SIZE)
```

```
# =====
```

```
# 2. Generator Model
```

```
# =====
```

```
def make_generator_model():
```

```
    model = tf.keras.Sequential([
```

```
        layers.Dense(  
            7 * 7 * 256,  
            use_bias=False,  
            input_shape=(100,) ,  
        ),
```

```
        layers.BatchNormalization(),
```

```
        layers.LeakyReLU(),
```

```
layers.Reshape(
    (7, 7, 256)
),

# 7x7 -> 7x7
layers.Conv2DTranspose(
    128,
    (5, 5),
    strides=(1, 1),
    padding="same",
    use_bias=False
),

layers.BatchNormalization(),
layers.LeakyReLU(),

# 7x7 -> 14x14
layers.Conv2DTranspose(
    64,
    (5, 5),
    strides=(2, 2),
    padding="same",
    use_bias=False
),

layers.BatchNormalization(),
layers.LeakyReLU(),
```

```

        # 14x14 -> 28x28
        layers.Conv2DTranspose(
            1,
            (5, 5),
            strides=(2, 2),
            padding="same",
            use_bias=False,
            activation="tanh"
        )
    ])

```

```

    return model

```

```

# =====
# 3. Discriminator Model
# =====

```

```

def make_discriminator_model():

```

```

    model = tf.keras.Sequential([

```

```

        # 28x28 -> 14x14
        layers.Conv2D(
            64,
            (5, 5),
            strides=(2, 2),

```

```
        padding="same",
        input_shape=[28, 28, 1]
    ),
```

```
layers.LeakyReLU(),
layers.Dropout(0.3),
```

```
# 14x14 -> 7x7
layers.Conv2D(
    128,
    (5, 5),
    strides=(2, 2),
    padding="same"
),
```

```
layers.LeakyReLU(),
layers.Dropout(0.3),
```

```
layers.Flatten(),
```

```
# Real or Fake
layers.Dense(1)
```

```
])
```

```
return model
```

```
# =====
```

4. Loss Functions

```
# =====
```

```
cross_entropy = tf.keras.losses.BinaryCrossentropy(  
    from_logits=True  
)
```

```
def generator_loss(fake_output):  
  
    return cross_entropy(  
        tf.ones_like(fake_output),  
        fake_output  
    )
```

```
def discriminator_loss(  
    real_output,  
    fake_output  
):  
  
    real_loss = cross_entropy(  
        tf.ones_like(real_output),  
        real_output  
    )  
  
    fake_loss = cross_entropy(  
        tf.zeros_like(fake_output),  
        fake_output
```

```

    )

    return real_loss + fake_loss

# =====
# 5. Create Models
# =====

generator = make_generator_model()

discriminator = make_discriminator_model()

# Optimizers
generator_optimizer = tf.keras.optimizers.Adam(
    0.0001
)

discriminator_optimizer = tf.keras.optimizers.Adam(
    0.0001
)

# =====
# 6. Training Settings
# =====

EPOCHS = 20

```

```

noise_dim = 100

num_examples = 16

seed = tf.random.normal(
    [num_examples, noise_dim]
)

# =====
# 7. Training Step
# =====

@tf.function
def train_step(images):

    # Generate random noise
    noise = tf.random.normal(
        [BATCH_SIZE, noise_dim]
    )

    with tf.GradientTape() as gen_tape, \
        tf.GradientTape() as disc_tape:

        # Generate fake images
        generated_images = generator(
            noise,
            training=True

```

```
)
```

```
# Discriminator predictions
```

```
real_output = discriminator(  
    images,  
    training=True  
)
```

```
fake_output = discriminator(  
    generated_images,  
    training=True  
)
```

```
# Calculate losses
```

```
gen_loss = generator_loss(  
    fake_output  
)
```

```
disc_loss = discriminator_loss(  
    real_output,  
    fake_output  
)
```

```
# Calculate gradients
```

```
gen_gradients = gen_tape.gradient(  
    gen_loss,  
    generator.trainable_variables
```

```

)

disc_gradients = disc_tape.gradient(
    disc_loss,
    discriminator.trainable_variables
)

# Update generator
generator_optimizer.apply_gradients(
    zip(
        gen_gradients,
        generator.trainable_variables
    )
)

# Update discriminator
discriminator_optimizer.apply_gradients(
    zip(
        disc_gradients,
        discriminator.trainable_variables
    )
)

# =====
# 8. Generate and Save Images
# =====

```

```
def generate_and_save_images(
    model,
    epoch,
    test_input
):

    predictions = model(
        test_input,
        training=False
    )

    plt.figure(figsize=(4, 4))

    for i in range(
        predictions.shape[0]
    ):

        plt.subplot(4, 4, i + 1)

        plt.imshow(
            predictions[i, :, :, 0] * 127.5 + 127.5,
            cmap="gray"
        )

        plt.axis("off")

    plt.savefig(
```

```

        f"image_at_epoch_{epoch:04d}.png"
    )

plt.close()

# =====
# 9. Train GAN
# =====

def train(dataset, epochs):

    for epoch in range(
        1,
        epochs + 1
    ):

        for image_batch in dataset:

            train_step(image_batch)

        # Generate image every 5 epochs
        if epoch % 5 == 0:

            generate_and_save_images(
                generator,
                epoch,
                seed

```

```

        )

        print(
            "Epoch:",
            epoch,
            "completed"
        )

# Start training
train(
    dataset,
    EPOCHS
)

# =====
# 10. Display Final Generated Images
# =====

final_images = generator(
    seed,
    training=False
)

plt.figure(figsize=(6, 6))

for i in range(16):

```

```
plt.subplot(4, 4, i + 1)
```

```
plt.imshow(  
    final_images[i, :, :, 0] * 127.5 + 127.5,  
    cmap="gray"  
)
```

```
plt.axis("off")
```

```
plt.tight_layout()
```

```
plt.show()
```